Optim.jl

CSCI 3656 - Anja Delzell

What key problems are solved?

The key problems solved by this software is Optimization problems. Some of the things this library does is listed below but it does much, much more: Minimizing functions, Solving Gradients and Hessians, Automatic differentiation, Conjugate Gradient, Gradient Descent, Momentum Gradient Descent, Newton

This software seems like a general purpose software in the sense that it solves many different types of optimization problems, but I believe the focus is optimization. **It uses Julia as a coding language,** so is specific to that. According to the read me "most of the solvers try to find an x that minimizes a function f(x) without any constraints." **The main focus of the software is on unconstrained optimization.** Most of the solvers included only converge to a local minimum, but there are a couple of methods available if a global minimum is required.

This package is unique because there are not many solvers written in Julia and it is really easy to add this package to your project to use it, instead of having to write your own optimization algorithm in julia. Because the solvers are written in Julia you can use multiple dispatch, which is where julia chooses which function implementation to use based on the types of inputs passed into the function. Working with a package in created in another language would minimize this benefit. One goal of there software is to make it so when doing an optimization problem one can use a "controlled choice of solvers for various steps in the algorithm."

Who are the stakeholders?

The individuals on the copyright are Patrick Kofod Mogensen, Asbjørn Nilsen Riseth, John Myles White, Tim Holy, which indicates to me they are some of the larger contributors of this software. Patrick Kofod Mogensen and David Müller-Widmann have contributed to the package most recently, both within the past week. **Optim.jl has 123 contributors, 168 branches, 19 open pull requests, 714 closed pull requests, 97 issues, and 490 closed issues.** I could not find any information about someone paying for Optim.jl

It seems like the developers are trying to make it so that people can easily run optimization solvers within julia, with a package designed for julia.

The creators collaborate through the github and communicate via multiple methods. If you have questions on the package you can ask them via Optim.jl's slack channel, chat with the creators on glitter, or go to the optimization section of the Julia discourse. It seems like glitter is the best way to reach the creators if you want to get involved with contributing to the package.

The github repository has 1.2k stars. **It seems like the software is probably used by people working in julia trying to solve problems other than optimization who want a quick way to implement optimization to continue solving whatever problem they are working on.**

It seems to me that this would positively impact people's time by making it easier to implement these things, and even help people understand how to make certain optimization solvers.

Metrics and features

Accuracy and cost are both factors that the creators considered when making their package. They store arrays in ways that would reduce the cost to them and the package does not compute certain hessians because they are not accurate enough: "Note that Optim will not generate approximate Hessians using finite differencing because of the potentially low accuracy of approximations to the Hessians. Other than Newton's method, none of the algorithms provided by the Optim package employ exact Hessians."

GradientDescent, ConjugateGradient and LBFGS methods support preconditioning which makes improved convergence possible. Preconditioning speeds up certain examples given in the readme as well.

The readme contains very specific instructions on how to use most individual functions and the limitation of the library. For example it is possible to access a minimizer even if all convergence flags are false so its recommended users include checks of outputs. **The speed of these functions isn't explicitly discussed in the readme but there is ways to put both iteration and time limits in the library so that tasks complete sooner (with less accuracy).**

Metrics and Features (continued)

I believe each algorithm probably has different levels of accuracy but I believe it would be possible to look at an algorithms accuracy vs cost. Stakeholders may want different things based on what their jobs are. Some people may want algorithms that run very quickly and others algorithms that are very precise. Maybe you could make your choice of algorithm you use in this library based on your goals.

There seem to be algorithmic choices made in the interest of good conditioning and stability depending on the algorithm you are looking at, however I didn't find much about these topics other than preconditioning in the readme. However, many of these algorithm pages reference different textbooks, which may have more information on specifics like stability or conditioning.

Optim Testing Summary

Test Summary:	Pass	Broken	Total	Time
Optim.jl	215050	73	215123	12m51.0s
special			None	9.9s
general	2300		2300	40.7s
univariate	56		56	8.5s
multivariate	211999	73	212072	5m27.5s
Literate examples	20		20	7.3s
show method for options	1		1	0.1s
MOI wrapper	674		674	6m17.1s

Testing Optim tests passed

I was rather confused on this. It says that the tests passed but then it said 73 tests were broken. This is a potential issue with this software

One Question I have About the Software

- Is there a way to graph convergence for these optimization problems within this library. We did graphing for gradient descent and I think being able to graph these problems would make them more understandable. However, I didn’t find anything in the documentation for graphing.

Proposed Experiment

I think it would be interesting to make some type of graph tool that works for multiple different algorithms. Something you could connect to this package so for each optimization problem you run you can easily graph the path the algorithm takes. Maybe you could compare different algorithms or different values of constraints on the graph too.

I think this would be presented by comparing the graphs built. We would need to find a way to check if they were accurate. Maybe create a manually built graph instead of this graph tool and compare them.